



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

گزارش آزمایش نهم

اعضای گروه

کیارش شجاعی ارانی

سینا فعالیت

پارسا ملکیان

نام درس

آزمایشگاه طراحی سامانه‌های رقمی

نیم‌سال تابستان ۱۴۰۵

نام استاد درس

دکتر اجلالی

۱ مقدمه

در حافظه‌ی معمولی، یک آدرس برای خواندن محتوای محل مشخصی ارائه می‌شود. در حافظه‌ی انجمنی^۱، جهت جست‌وجو برعکس است: داده جست‌وجو می‌شود و حافظه مشخص می‌کند کدام ورودی‌ها با آن تطبیق دارند. این ویژگی برای مسائلی مناسب است که تصمیم باید بر اساس محتوای کلید و نه آدرس از پیش معلوم انجام شود.

TCAM^۲ نوع توسعه‌یافته‌ی CAM است. در CAM معمولی هر بیت فقط صفر یا یک است، اما در TCAM حالت سوم X نیز وجود دارد. X به معنای بی‌تفاوت است و بیت متناظر کلید جست‌وجو را از مقایسه حذف می‌کند. صورت آزمایش طراحی یک TCAM شامل ۱۶ ثبات ۱۶ بیتی را می‌خواهد و تأکید می‌کند که طرح باید پارامتری باشد.

۲ تعریف تطبیق سه‌حاله

برای هر بیت یک ورودی ذخیره‌شده، دو مقدار نگه‌داری می‌شود:

- `stored_data` مقدار صفر یا یک پایه را نگه می‌دارد؛
- `stored_x_mask` مشخص می‌کند کدام موقعیت‌ها X هستند؛
- اگر بیت ماسک صفر باشد، بیت داده باید با `search_data` برابر باشد؛
- اگر بیت ماسک یک باشد، نتیجه‌ی آن موقعیت مستقل از `search_data` برابر در نظر گرفته می‌شود.

بیت‌هایی که تفاوت دارند با استفاده از رابطه زیر به دست می‌آیند:

$$\text{mismatched_bits} = (\text{search_data} \oplus \text{stored_data}) \wedge \neg \text{stored_x_mask} \quad (۱)$$

ورودی زمانی تطبیق دارد که معتبر باشد:

^۱Content Addressable Memory

^۲Ternary Content Addressable Memory

$$\text{match} = \text{valid} \wedge (\text{mismatched_bits} = 0) \quad (2)$$

این نمایش دو مزیت دارد. نخست آنکه X به عنوان مقدار ناشناخته‌ی شبیه‌ساز وارد مدار سنتزپذیر نمی‌شود. دوم آنکه قاعده‌ی مقایسه با چند عمل منطقی ساده قابل پیاده‌سازی است.

۳ مشخصات و رابط طرح

هر دو پیمانه‌ی tcam_no_loop و tcam دو پارامتر دارند:

• DATA_WIDTH : عرض هر ورودی، با مقدار پیش‌فرض ۱۶؛

• DEPTH : تعداد ورودی‌ها، با مقدار پیش‌فرض ۱۶.

جدول ۱ نقش سیگنال‌های خارجی را خلاصه می‌کند.

جدول ۱: رابط خارجی پیمانه‌ی TCAM

سیگنال	جهت	نقش
clk	ورودی	ساعت ثبت داده و ماسک
reset	ورودی	بازنشانی فعال‌بالا و هم‌زمان برای بیت‌های اعتبار
write_enable	ورودی	بردار DEPTH بیتی؛ مجوز مستقل هر ورودی
write_data	ورودی	بردار داده با عرض DEPTH ضرب در DATA_WIDTH
write_x_mask	ورودی	بردار ماسک با همان عرض؛ یک بخش مستقل برای هر ورودی
search_data	ورودی	کلید مشترک جست‌وجو با عرض DATA_WIDTH
match_lines	خروجی	یک بیت نتیجه برای هر ورودی

در پیکربندی پیش‌فرض، match_lines شانزده‌بیتی است. بیت i این بردار مستقیماً نتیجه‌ی ورودی i را نشان می‌دهد. اولویت‌گذاری میان تطبیق‌ها انجام نمی‌شود؛ بنابراین چند بیت می‌توانند هم‌زمان یک باشند. این رفتار برای مثال صورت آزمایش ضروری است، زیرا یک داده باید با سه الگوی متفاوت تطبیق پیدا کند.

۴ معماری کلی

طرح از یک پیمانه‌ی ذخیره‌سازی و سه پیاده‌سازی سطح بالا تشکیل شده است:

- `tcam_entry.v`: ذخیره‌ی داده، ماسک و بیت اعتبار یک ورودی و تولید نتیجه‌ی همان ورودی؛

- `tcam.v`: ساخت تعداد پارامتری از `tcam_entry`ها با حلقه‌ی `generate`؛

- `tcam_no_loop.v`: اتصال همان ورودی‌ها با آرایه‌ی نمونه‌ها و بدون حلقه‌ی `for`؛

- `tcam_16x16_top.v`: سطح بالای آدرس‌دار برای حفظ ظرفیت 16×16 با تعداد پایه‌ی قابل‌استفاده روی بورد. این طراحی به این منظور پیاده‌سازی شد که دو طراحی قبلی با ابعاد بالا قابل‌سنتز نبودند.

هر ورودی، بیت مجوز و بخش داده و ماسک مستقل خود را دریافت می‌کند. بنابراین چند ورودی می‌توانند در یک لبه‌ی ساعت، مقادیر متفاوت بگیرند. `search_data` به همه‌ی ورودی‌ها متصل است و مقایسه‌ها به‌صورت موازی انجام می‌شوند.

۵ طراحی یک ورودی TCAM

هر `tcam_entry` سه ثبات `stored_data`، `stored_x_mask` و `valid` دارد. هنگام `reset` فقط `valid` صفر می‌شود. پاک‌نکردن فیزیکی داده و ماسک مشکلی ایجاد نمی‌کند، زیرا `valid` صفر، خروجی `match` را بدون توجه به محتوای قدیمی غیرفعال نگه می‌دارد. این تصمیم منطق بازنشانی را کاهش می‌دهد و از پاک‌سازی غیرضروری تمام بیت‌های ذخیره جلوگیری می‌کند.

خطوط ۲۳ تا ۳۵ پرونده‌ی `tcam_entry.v` منطق نوشتن، ماسک‌کردن و تطبیق را نشان می‌دهند.

```

23 always @(posedge clk) begin
24     if (reset) begin
25         valid <= 1'b0;
26     end else if (write_enable) begin
27         stored_data <= write_data;
28         stored_x_mask <= write_x_mask;
29         valid <= 1'b1;
30     end
31 end
32
33 // Masked positions are removed before checking for unequal bits.
34 assign mismatched_bits = (search_data ^ stored_data) & ~stored_x_mask;
35 assign match = valid && (mismatched_bits == {DATA_WIDTH{1'b0}});

```

هنگام فعال بودن write_enable در لبه‌ی بالارونده، داده و ماسک جدید ذخیره و ورودی معتبر می‌شود. سپس منطق ترکیبی اختلاف بیت‌ها را محاسبه می‌کند. ابتدا XOR تفاوت search_data و stored_data را مشخص می‌کند؛ سپس معکوس ماسک فقط بیت‌های مهم را نگه می‌دارد. صفرشدن کل بردار اختلاف به معنای تطبیق کامل همه‌ی موقعیت‌های غیر X است.

۶ طراحی پیمانه‌ی سطح بالا

هر دو پیمانه از tcam_entry استفاده می‌کنند. هر بیت فعال write_enable، نوشتن ورودی متناظر را مجاز می‌کند. اگر بازنشانی فعال نباشد، ورودی‌های غیرفعال داده، ماسک و اعتبار قبلی خود را حفظ می‌کنند.

عرض هرکدام از write_data و write_x_mask برابر حاصل ضرب تعداد ورودی‌ها در عرض هر ورودی است. برش ورودی با شماره i از بیت حاصل ضرب i در عرض داده آغاز می‌شود؛ ورودی صفر در کم‌ارزش‌ترین بخش قرار می‌گیرد. برای چهار ثبات سه‌بیتی، اتصال {3'b111, 3'b100, 3'b010, 3'b001} به write_data، به ترتیب داده‌های 001، 010، 100 و 111 را به ورودی‌های صفر تا سه می‌رساند. با مجوز 1111 هر چهار مقدار در همان لبه ثبت می‌شوند؛ با مجوز 1010 فقط ورودی‌های یک و سه تغییر می‌کنند. کلید search_data همچنان مشترک است و همه تطبیق‌ها در match_lines دیده می‌شوند.

در tcam.v حلقه ساخت، برش داده، برش ماسک و بیت مجوز هر ورودی را به نمونه مربوط وصل می‌کند. در tcam_no_loop.v آرایه نمونه‌ها همین اتصال را بدون for می‌سازد. بردارهای داده و ماسک میان نمونه‌ها تقسیم می‌شوند، ولی ساعت، بازنشانی و کلید جست‌وجو مشترک می‌مانند. این دو شیوه رفتار خارجی یکسان دارند؛ حلقه ساخت نیز مراحل زمانی پیاپی ایجاد نمی‌کند.

اتصال‌های نسخه حلقه‌دار در خطوط ۲۵ تا ۳۵ پرونده tcam.v چنین است:

```
25 for (row = 0; row < DEPTH; row = row + 1) begin : entries
26     tcam_entry #(.DATA_WIDTH(DATA_WIDTH)) entry (
27         .clk(clk),
28         .reset(reset),
29         .write_enable(write_enable[row]),
30         .write_data(write_data[row*DATA_WIDTH +: DATA_WIDTH]),
31         .write_x_mask(write_x_mask[row*DATA_WIDTH +: DATA_WIDTH]),
32         .search_data(search_data),
33         .match(match_lines[row])
34     );
35 end
```

آرایه نمونه‌ها^۳ در خطوط ۲۲ تا ۳۰ پرونده tcam_no_loop.v همان اتصال را بدون حلقه بیان می‌کند:

```
22 tcam_entry #(.DATA_WIDTH(DATA_WIDTH)) entries [DEPTH-1:0] (
23     .clk(clk),
24     .reset(reset),
25     .write_enable(write_enable),
26     .write_data(write_data),
27     .write_x_mask(write_x_mask),
28     .search_data(search_data),
29     .match(match_lines)
30 );
```

^۳Instance array

۱-۶ محدوده پارامترها و حالت صفربیتی

محدوده معتبر هر دو پیمان، عرض داده و تعداد ورودی بزرگتر یا مساوی یک است. پیکربندی یک ثبات صفربیتی عمداً رد می‌شود. عبارت [1:0] در Verilog بردار صفربیتی نیست، بلکه یک بازه دوییتی است؛ بنابراین تفسیر آن به عنوان حافظه صفربیتی صحیح نیست. شاخه کنترل پارامترها پیش از ساخت ورودی‌های حافظه، پیکربندی نامعتبر را در زمان صفر با پیام مشخص رد می‌کند. کوچک‌ترین پیکربندی معتبر یک ثبات یک‌بیتی است.

۷ روش آزمون

هفت آزمون رفتاری برای مقایسه‌ی دو پیمانه‌ی حلقه‌دار و بدون حلقه و یک آزمون مستقل برای سطح بالای آدرس‌دار نوشته شد. هفت آزمون نخست هر دو پیمانه را با ورودی یکسان اجرا می‌کنند و خروجی هر کدام را جداگانه با مقدار مورد انتظار می‌سنجند.

آزمون پارامتر نامعتبر جداگانه اجرا می‌شود و معیار موفقیت آن توقف در زمان صفر با پیام کنترل ابعاد و کد خروج غیر صفر است.

۱-۷ آزمون الگوهای Wildcard صورت آزمایش

در tb_tcam_wildcard.v برای بازسازی مثال صورت آزمایش، عرض ۸ و عمق ۴ انتخاب شد. سه ورودی نخست در لبه‌ی ۱۵ نانوثانیه با الگوهای زیر هم‌زمان برنامه‌ریزی شدند:

• 0110XXXX؛

• X1101XXX؛

• 0X1X11X0.

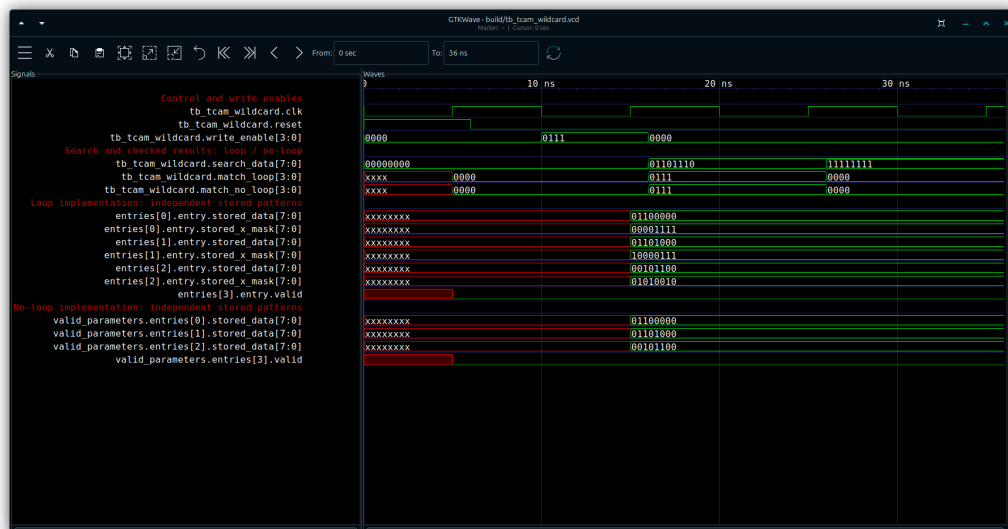
الگوها با ترکیب stored_data و stored_x_mask نمایش داده شدند. برای کلید 01101110، همه‌ی بیت‌های محدود شده در هر سه الگو برابر بودند؛ در نتیجه سه خط نخست فعال و match_lines برابر 0111 شد. مجوز ورودی چهارم صفر بود؛ حتی با وجود ماسک ورودی کاملاً بی‌تفاوت، این ورودی برنامه‌ریزی نشد و به دلیل valid صفر تطبیق نداشت. سپس کلید 11111111 اعمال شد و هیچ الگو با آن سازگار نبود؛ بنابراین خروجی به 0000 تغییر کرد.

خطوط ۵۰ تا ۵۹ پروندهی tb_tcam_wildcard.v بردارهای داده و ماسک مثال صورت آزمایش و دو نتیجهی مورد انتظار را پیاده‌سازی می‌کند.

```

50 // Rows 0..2: 0110XXXX, X1101XXX, 0X1X11X0.
51 // Row 3 stays invalid even though its input mask is all-X.
52 write_data = {8'b00000000, 8'b00101100, 8'b01101000, 8'b01100000};
53 write_x_mask = {8'b11111111, 8'b01010010, 8'b10000111, 8'b00001111};
54 write_enable = 4'b0111;
55 @(posedge clk);
56 #1 write_enable = 4'b0000;
57
58 check_search(8'b01101110, 4'b0111);
59 check_search(8'b11111111, 4'b0000);

```

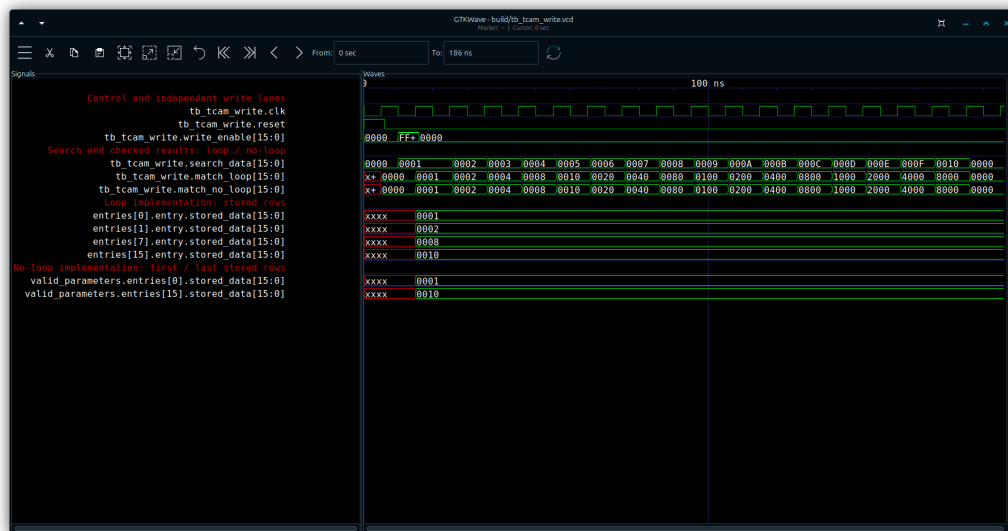


شکل ۱: تطبیق هم‌زمان کلید 01101110 با سه الگوی سه‌حالتی صورت آزمایش

در tb_tcam_wildcard.vcd، سه داده و ماسک متفاوت در لبه‌ی ۱۵ نانوثانیه ثبت می‌شوند. کلید 01101110 از زمان ۱۶ نانوثانیه در هر دو پیاده‌سازی خروجی 0111 ایجاد می‌کند. کلید 11111111 از زمان ۲۶ نانوثانیه خروجی را به 0000 تغییر می‌دهد؛ ورودی چهارم در تمام آزمون نامعتبر می‌ماند. بنابراین بیت X از مقایسه حذف می‌شود، اما بیت‌های صفر و یک ذخیره‌شده باید دقیقاً با کلید برابر باشند.

۲-۷ ثبت شانزده مقدار متفاوت در یک لبه ساعت

آزمون tb_tcam_write.v با پیکربندی ۱۶×۱۶، مقادیر مبنای شانزده 0001 تا 0010 را به ترتیب در ورودی‌های صفر تا ۱۵ قرار داد. تمام مجوزها در زمان ۱۰ نانوثانیه فعال شدند و همه داده‌ها در لبه ۱۵ نانوثانیه ثبت شدند. پیش از این لبه هیچ تطبیقی مجاز نبود. سپس هر شانزده مقدار جداگانه جست‌وجو شد و فقط بیت متناظر فعال شد؛ جست‌وجوی صفر نیز هیچ تطبیقی نداشت.



شکل ۲: ثبت هم‌زمان شانزده داده متفاوت و جست‌وجوی مستقل آن‌ها

در این شکل از tb_tcam_write.vcd، داده‌های متفاوت ورودی‌های صفر، یک، هفت و پانزده در یک لبه ثبت شده‌اند. نتایج match_loop و match_no_loop برای همه کلیدها یکسان‌اند و از 0001 تا 8000 پیش می‌روند؛ در پایان هر دو صفر می‌شوند.

۳-۷ شانزده ثبات شانزده‌بیتی با ماسک‌های مستقل و نوشتن انتخابی

آزمون tb_tcam_16x16.v با عرض ۱۶ و تعداد ۱۶، هر دو پیاده‌سازی را در ۷۲ جست‌وجو بررسی کرد. در ورودی‌های صفر تا ۱۵، مقادیر مبنای شانزده 0000، 1111، 2222 و به همین ترتیب تا FFFF قرار گرفتند. ماسک ورودی i فقط بیت i را یک می‌کند؛ بنابراین ماسک‌ها به ترتیب 0001، 0002، 0004 و تا 8000 هستند. داده‌ها و ماسک‌ها در یک لبه، در زمان ۱۵ نانوثانیه ثبت شدند. پیش از این لبه، هیچ ورودی نباید تطبیق تولید کند.

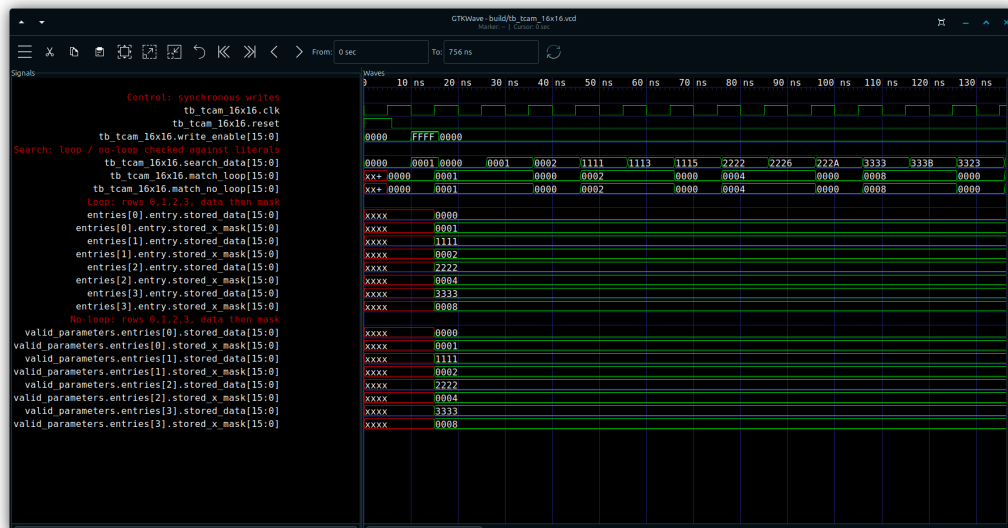
خطوط ۶۵ تا ۷۴ پرونده‌ی tb_tcam_16x16.v داده‌ها و ماسک‌های مستقل همه‌ی ورودی‌ها را مشخص می‌کنند:

```

65 // Row 0 is the rightmost word. Each row ignores a different bit.
66 write_data = {16'hFFFF, 16'hEEEE, 16'hDDDD, 16'hCCCC,
67               16'hBBBB, 16'hAAAA, 16'h9999, 16'h8888,
68               16'h7777, 16'h6666, 16'h5555, 16'h4444,
69               16'h3333, 16'h2222, 16'h1111, 16'h0000};
70 write_x_mask = {16'h8000, 16'h4000, 16'h2000, 16'h1000,
71                16'h0800, 16'h0400, 16'h0200, 16'h0100,
72                16'h0080, 16'h0040, 16'h0020, 16'h0010,
73                16'h0008, 16'h0004, 16'h0002, 16'h0001};
74 write_enable = 16'hFFFF;

```

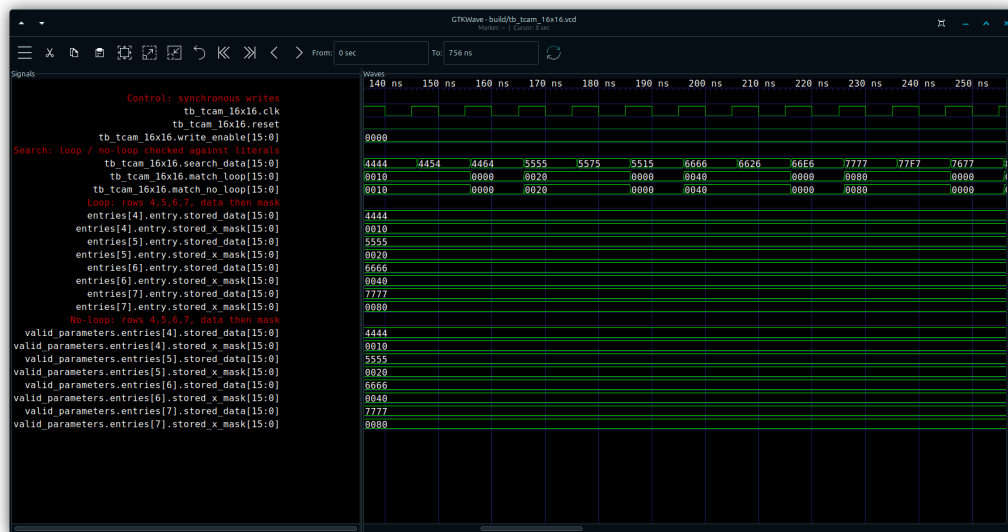
برای هر ورودی سه کلید ثابت بررسی شد: داده اصلی، داده با تغییر بیت ماسک شده و داده با تغییر یک بیت غیرماسک شده. دو کلید نخست فقط خط همان ورودی را فعال می کنند، اما کلید سوم هیچ تطبیقی ندارد. این بخش ۴۸ جست و جو دارد و همه موقعیت های ماسک، از بیت صفر تا بیت ۱۵، را پوشش می دهد. مقادیر مورد انتظار به صورت ثابت در آزمون نوشته شده اند و از منطق مقایسه پیمانه محاسبه نمی شوند. همه مقدارهای داده، ماسک و نتیجه در تصاویر این بخش بر حسب مبنای شانزده هستند.



شکل ۳: ماسک های مستقل ورودی های صفر تا سه در پیکربندی شانزده در شانزده

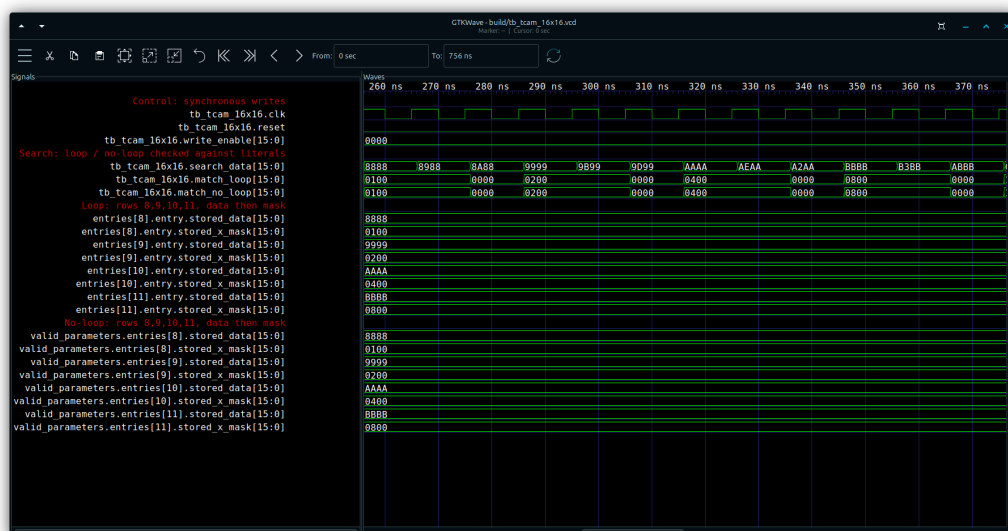
نمای ورودی های صفر تا سه از `tb_tcam_16x16.vcd`، ثبت هم زمان چهار جفت داده و ماسک را در لبه ۱۵ نانوثانیه نشان می دهد. برای نمونه، 0000 و 0001 هر دو خروجی 0001 می دهند، اما

0002 خروجی صفر دارد. داده هر ورودی بلافاصله بالای ماسک همان ورودی قرار دارد و هر دو نسخه همین ترتیب را دارند.



شکل ۴: تطبیق دقیق، بی تفاوت و عدم تطبیق ورودی‌های چهار تا هفت

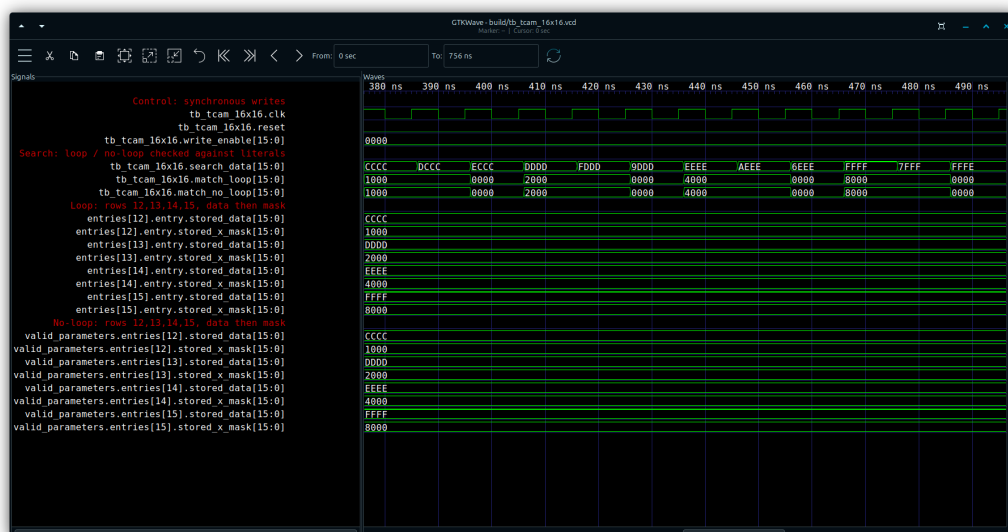
در بازه ۱۳۶ تا ۲۵۶ نانوثانیه، ورودی‌های چهار تا هفت با ماسک‌های 0010، 0020، 0040 و 0080 بررسی می‌شوند. برای ورودی هفت، کلیدهای 7777 و 77F7 خروجی 0080 دارند و 7677 تطبیق ندارد. ثابت ماندن داده و ماسک هنگام تغییر کلید، ترکیبی بودن جست‌وجو را نشان می‌دهد.



شکل ۵: بررسی ماسک‌های نیمه بالایی کلمه در ورودی‌های هشت تا یازده

این نما در بازه ۲۵۶ تا ۳۷۶ نانوثانیه، ماسک‌های 0100، 0200، 0400 و 0800 را برای ورودی‌های هشت تا یازده نمایش می‌دهد. برای نمونه، 8888 و 8988 هر دو خط هشت را فعال می‌کنند، ولی

8A88 خروجی صفر می‌دهد. بنابراین بیت‌های ماسک در نیمه بالایی کلمه نیز به درستی عمل می‌کنند.



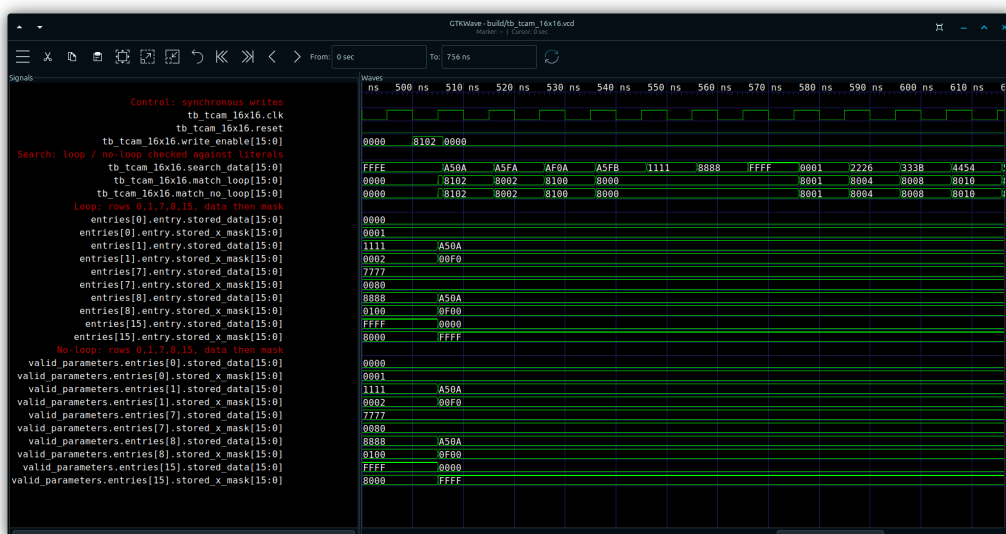
شکل ۶: بررسی ورودی‌های دوازده تا پانزده و پرازش‌ترین بیت ماسک

در بازه ۳۷۶ تا ۴۹۶ نانوثانیه، داده و ماسک ورودی‌های دوازده تا پانزده به ترتیب نمایش داده شده‌اند. ماسک آخرین ورودی 8000 است؛ در نتیجه FFFF و 7FFF هر دو خروجی 8000 دارند، اما FFFE تطبیق ندارد. این بخش، آخرین ورودی و پرازش‌ترین بیت ماسک را در هر دو نسخه بررسی می‌کند.

در مرحله دوم، ورودی‌های داده و ماسک همه ثبات‌ها در زمان ۵۰۰ نانوثانیه تغییر کردند، اما `write_enable` برابر 8102 بود؛ یعنی فقط ورودی‌های یک، هشت و پانزده مجوز نوشتن داشتند. در لبه ۵۰۵ نانوثانیه، ورودی یک داده A50A و ماسک 00F0، ورودی هشت داده A50A و ماسک 0F00، و ورودی پانزده داده 0000 و ماسک FFFF را ذخیره کردند. ماسک آخرین ورودی تمام بیت‌ها را بی تفاوت می‌کند. داده و ماسک سیزده ورودی غیرفعال باید دست‌نخورده بمانند.

نتایج اصلی این مرحله چنین بودند:

- A50A: خروجی 8102، تطبیق هم‌زمان ورودی‌های یک، هشت و پانزده؛
- A5FA: خروجی 8002، تطبیق ورودی‌های یک و پانزده؛
- AF0A: خروجی 8100، تطبیق ورودی‌های هشت و پانزده؛
- A5FB: خروجی 8000، تطبیق فقط با ورودی کاملاً بی تفاوت.



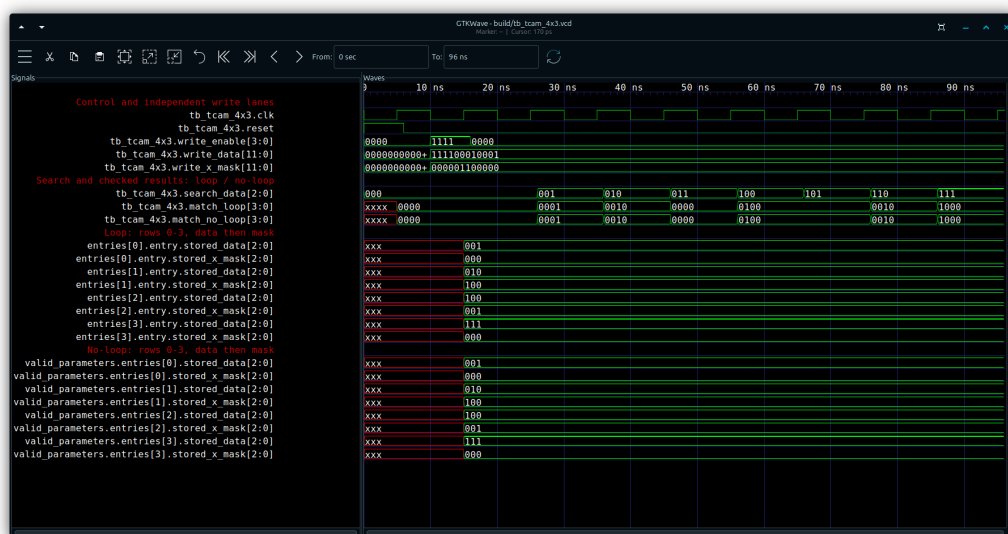
شکل ۷: نوشتن انتخابی در ورودی‌های یک، هشت و پانزده و حفظ ورودی‌های غیرفعال

این نما از tb_tcam_16x16.vcd مجوز 8102 و تغییر داده و ماسک ورودی‌های یک، هشت و پانزده در لبه ۵۰۵ نانوثانیه را نشان می‌دهد. جفت داده و ماسک ورودی‌های صفر و هفت نیز برای مقایسه نمایش داده شده‌اند و ثابت می‌مانند. نتایج دو پیاده‌سازی، تطبیق هم‌زمان چند ورودی و تفاوت اثر دو ماسک 00F0 و 0F00 را نشان می‌دهند.

پس از نوشتن انتخابی، هفت جست‌وجو برای ورودی‌های بازنویسی‌شده و سیزده جست‌وجو برای حفظ ورودی‌های غیرفعال انجام شد. برای نمونه، 0001 خروجی 8001 و 77F7 خروجی 8080 داد؛ پس ورودی‌های صفر و هفت، با وجود تغییر ورودی خارجی خود، داده و ماسک قبلی را حفظ کردند. در پایان، همه داده‌ها و ماسک‌های ورودی با مجوز نوشتن صفر تغییر کردند و چهار جست‌وجوی دیگر پس از لبه ۷۱۵ نانوثانیه همچنان نتایج قبلی را دادند. مجموع ۴۸، ۷، ۱۳ و ۴ جست‌وجو برابر ۷۲ است؛ آزمون در زمان ۷۵۶ نانوثانیه با پیام PASS پایان یافت.

۴-۷ چهار ثبات سه‌بیتی با ماسک‌های مستقل

آزمون tb_tcam_4x3.v از عرض ۳ و عمق ۴ استفاده کرد. ورودی‌های صفر تا سه به‌ترتیب الگوهای 10X، X10، 001 و 111 را در یک لبه دریافت کردند. هر هشت کلید ممکن سه‌بیتی بررسی شد. برای کلیدهای 000، 001، 010، 011، 100، 101، 110 و 111، بردارهای مورد انتظار به‌ترتیب 0000، 0001، 0010، 0011، 0100، 0101، 1000 و 1001 بودند. هر دو نسخه تمام این نتایج را تولید کردند.

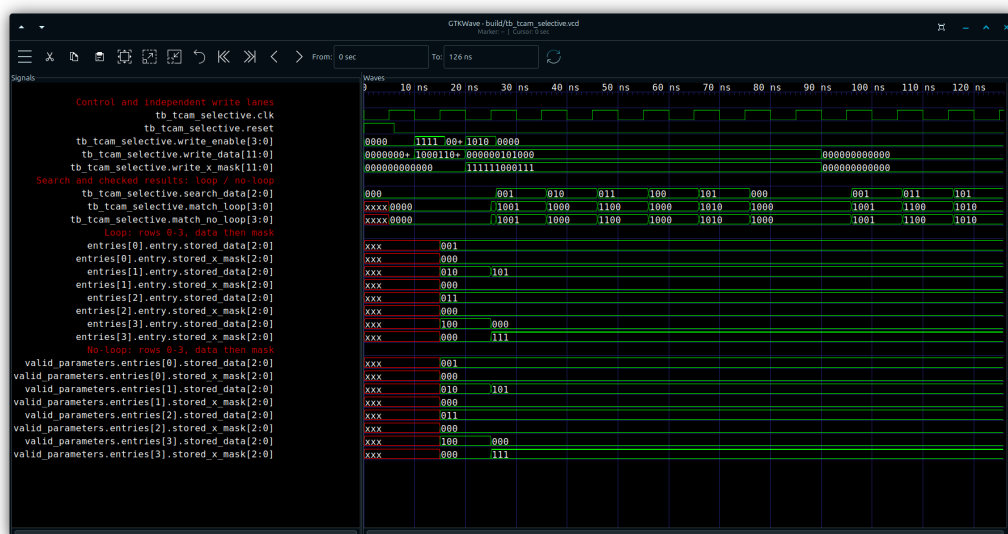


شکل ۸: پیکربندی چهار ثبات سه‌بیتی و بررسی همه کلیدهای ممکن

شکل از tb_tcam_4x3.vcd ثبت هم‌زمان چهار داده و ماسک متفاوت ورودی‌های یک و دو را نشان می‌دهد. تغییر کلید بدون نوشتن دوباره، همه حالت‌های جست‌وجوی سه‌بیتی را طی می‌کند و خروجی دو پیاده‌سازی در هر مرحله با مقدار مورد انتظار برابر است.

۷-۵ نوشتن انتخابی و حفظ ورودی‌های غیرفعال

در tb_tcam_selective.v ابتدا داده‌های 001، 010، 011 و 100 در چهار ورودی نوشته شدند. سپس با مجوز 1010، در یک لبه ورودی یک به 101 و ورودی سه به XXX تغییر کرد. داده و ماسک ورودی‌های صفر و دو با وجود تغییر ورودی خارجی آن‌ها ثابت ماندند. جست‌وجوی 001 نتیجه 1001، جست‌وجوی 011 نتیجه 1100 و جست‌وجوی 101 نتیجه 1010 داد. مقادیر قدیمی 010 و 100 فقط با ورودی کاملاً بی‌تفاوت تطبیق داشتند. تغییر همه داده‌های نوشتن با مجوز صفر نیز هیچ ورودی را بازنویسی نکرد.



شکل ۹: بهروزرسانی همزمان ورودی‌های یک و سه و حفظ ورودی‌های صفر و دو

در `tb_tcam_selective.vcd`، لبه ۲۵ نانوثانیه فقط داده ورودی‌های یک و سه را تغییر می‌دهد. ماسک ورودی سه 111 می‌شود، اما داده و ماسک ورودی‌های صفر و دو ثابت می‌مانند. چندتطبیقی بودن خروجی و حفظ داده هنگام صفرشدن تمام مجوزها در هر دو نسخه دیده می‌شود.

۶-۷ کوچک‌ترین اندازه معتبر و اولویت بازنشانی

آزمون `tb_tcam_1x1.v` یک ثابت یک‌بیتی را بررسی کرد. ورودی نامعتبر ابتدا با هیچ‌یک از دو کلید تطبیق نداشت. پس از ثبت یک با ماسک صفر در زمان ۳۵ نانوثانیه، فقط کلید یک تطبیق داشت. با ثبت ماسک یک در زمان ۶۵ نانوثانیه، هر دو کلید تطبیق پیدا کردند. بازنشانی در زمان ۹۰ نانوثانیه هم‌زمان با مجوز نوشتن فعال شد؛ خروجی تا لبه ۹۵ نانوثانیه معتبر ماند و سپس برای هر دو کلید صفر شد. بنابراین بازنشانی هم‌زمان است و نسبت به نوشتن اولویت دارد.

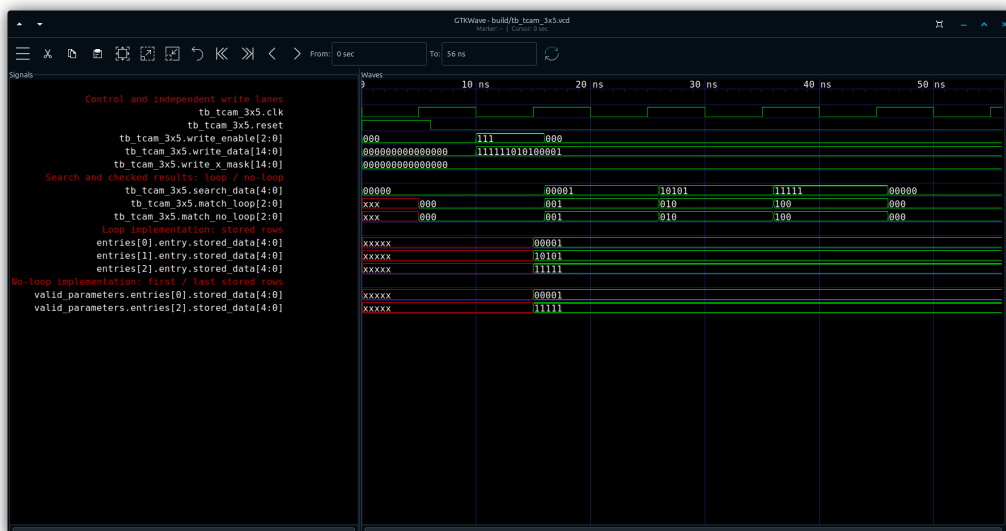


شکل ۱۰: آزمون یک ثابت یک بیتی، تطبیق بی تفاوت و اولویت بازنشانی

فایل `tb_tcam_1x1.vcd` گذار از ورودی نامعتبر به تطبیق دقیق، سپس الگوی X و در پایان بی اعتبار شدن در لبه بازنشانی را نشان می دهد. رفتار `valid` و نتیجه جست و جو در هر دو نسخه یکسان است.

۷-۷ تعداد ورودی غیرتوان دو

آزمون `tb_tcam_3x5.v` سه ثابت پنج بیتی با داده های 00001، 10101 و 11111 ساخت. این داده ها هم زمان نوشته شدند و جست و جوی آن ها به ترتیب خروجی های 001، 010 و 100 داد. کلید 00000 هیچ تطبیقی نداشت. این آزمون نشان می دهد تعداد ورودی ها لازم نیست توان دو باشد و مرزبندی داده ها برای عرض پنج بیت نیز صحیح است.



شکل ۱۱: نوشتن و جست و جوی سه ثبات پنج بیتی

فایل tb_tcam_3x5.vcd ثبت سه کلمه پنج بیتی در لبه ۱۵ نانوثانیه و فعال شدن مستقل سه خط تطبیق را در هر دو نسخه ثبت می کند.

۷-۸ رد یک ثبات صفر بیتی و ابعاد نامعتبر

آزمون tb_tcam_invalid.v در حالت پیش فرض عرض صفر و تعداد یک دارد. با NO_LOOP=0 نسخه حلقه دار و با NO_LOOP=1 نسخه بدون حلقه جداگانه بررسی شدند. هر دو در زمان صفر با کد خروج یک و پیام TCAM requires DATA_WIDTH >= 1 and DEPTH >= 1 متوقف شدند. حالت های عرض منفی، تعداد صفر و تعداد منفی نیز برای هر دو پیاده سازی رد شدند.

اگر پارامتر نامعتبر پذیرفته شود، خود آزمون در زمان یک نانوثانیه با پیام متفاوتی خطا می دهد؛ بنابراین صرف دریافت هر خطایی به معنای موفقیت نیست. این آزمون شکل موج رفتاری تولید نمی کند، زیرا پیکربندی نامعتبر پیش از شروع کار حافظه رد می شود.

```
tb_tcam_invalid.v: DEPTH=1, DATA_WIDTH=0
Loop implementation (tcam.v):
FATAL: tcam.v:22: TCAM requires DATA_WIDTH >= 1 and DEPTH >= 1
Time: 0 Scope: tb_tcam_invalid.test_loop.dut.invalid_parameters
Simulator exit status: 1 (expected nonzero)
No-loop implementation (tcam no_loop.v):
FATAL: tcam no_loop.v:20: TCAM requires DATA_WIDTH >= 1 and DEPTH >= 1
Time: 0 Scope: tb_tcam_invalid.test_no_loop.dut.invalid_parameters
Simulator exit status: 1 (expected nonzero)
Zero-width configurations are rejected at time 0.
```

شکل ۱۲: خروجی واقعی رد یک ثبات صفر بیتی در هر دو پیاده سازی

تصویر پایانه نشان می‌دهد هر دو پیاده‌سازی با پیام کنترل ابعاد در زمان صفر و کد خروج یک متوقف می‌شوند. فایل‌های `invalid-0x1-0.log` و `invalid-0x1-1.log` متن همین خطاها را نگه می‌دارند. این تصویر مدرک رد پارامتر است و شکل موج یک حافظه معتبر نیست.

۸ نتایج اجرای خودکار

هر هشت آزمون رفتاری با Icarus Verilog و گزینه‌های `-g2012` و `-Wall` کامپایل و اجرا شدند. هیچ هشدار کامپایل یا شکست آزمونی ثبت نشد. نتیجه‌ی همه‌ی آزمون‌ها PASS بود:

- `tb_tcam_write.v`: شانزده داده‌ی مستقل در پیکربندی 16×16 ؛
- `tb_tcam_16x16.v`: ۷۲ جست‌وجو با ماسک‌های مستقل شانزده‌بیتی و نوشتن انتخابی؛
- `tb_tcam_4x3.v`: ماسک مستقل و تمام کلیدهای پیکربندی 4×3 ؛
- `tb_tcam_selective.v`: نوشتن انتخابی و حفظ ورودی‌های غیرفعال؛
- `tb_tcam_1x1.v`: حداقل اندازه‌ی معتبر، تطبیق دقیق و `X`، و اولویت بازنشانی؛
- `tb_tcam_3x5.v`: تعداد غیرتوان دو و برش صحیح داده‌ی پنج‌بیتی؛
- `tb_tcam_wildcard.v`: سه الگوی صورت آزمایش در پیکربندی 4×8 ؛
- `tb_tcam_16x16_top.v`: نوشتن آدرس‌دار همه‌ی شانزده ورودی، ماسک، عدم تطبیق و بازنشانی سطح بالای مورد.

۹ نتایج سنتز در Quartus

هر دو پیمانه‌ی `tcam` و `tcam_no_loop` در پنج پیکربندی 16×16 ، 4×3 ، 1×1 ، 3×5 و 4×8 با Yosys و گذر `synth` و سپس `check -assert` بررسی شدند. هر ده اجرا بدون مشکل پایان یافت. برای تکمیل بررسی مستقل، سه سطح بالای `tcam`، `tcam_no_loop` و `tcam_16x16_top` با Quartus II 13.0.1 برای خانواده‌ی Cyclone IV GX تحلیل و سنتز شدند. دو نسخه‌ی نخست با پارامترهای `DATA_WIDTH=4` و `DEPTH=4` بررسی شدند؛ بنابراین تصاویر مربوط به آن‌ها یک ساختار

چهار ورودی در چهار بیت را نشان می‌دهند، نه یک ساختار دو در دو. نسخه‌ی سوم پیکربندی کامل شانزده ورودی شانزده‌بیتی است.

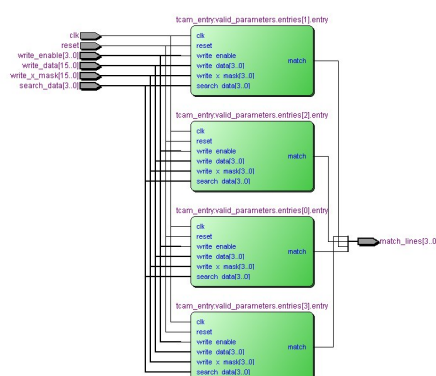
۹-۱ سنتر نسخه‌ی حلقه‌دار در پیکربندی چهار در چهار

شکل ۱۳ تنظیم صریح هر دو پارامتر روی مقدار چهار را ثبت می‌کند. این تنظیم باعث می‌شود چهار ورودی مستقل، هرکدام با داده و ماسک چهاربیتی، در زمان تشریح ساخته شوند و امکان مقایسه‌ی منصفانه‌ی دو شیوه‌ی ساخت فراهم شود.

	Parameter Name	Value	Type
1	DATA_WIDTH	4	Signed Integer
2	DEPTH	4	Signed Integer

شکل ۱۳: پارامترهای عرض داده و تعداد ورودی برای سنتر چهار در چهار

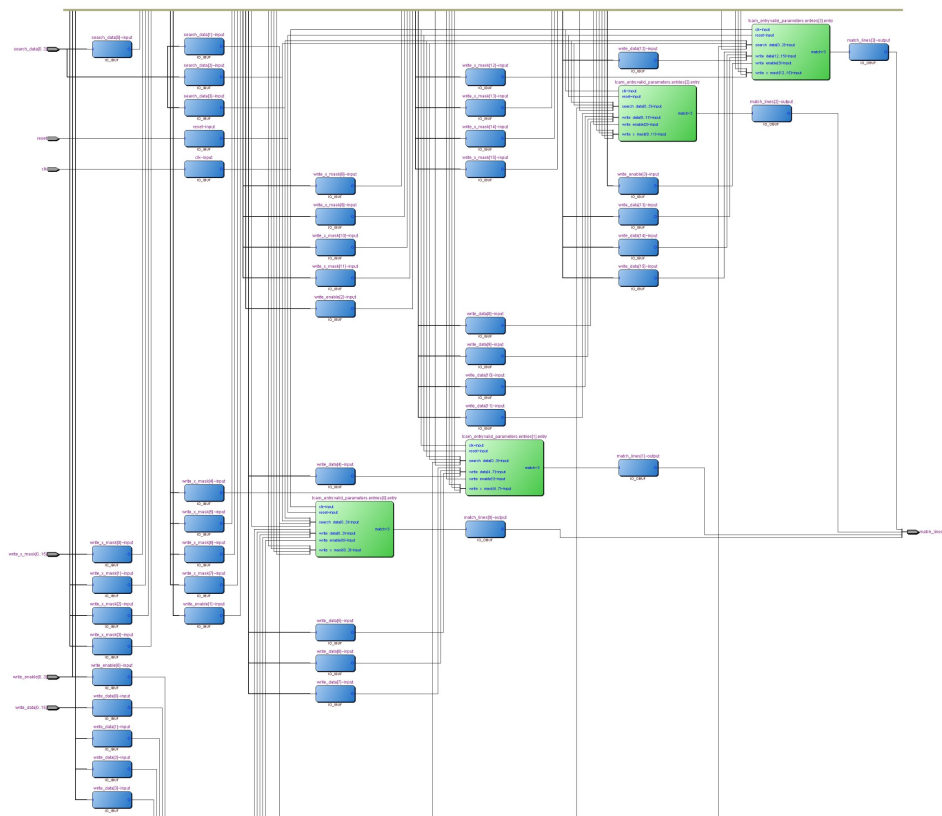
شکل ۱۴ نتیجه‌ی تشریح حلقه‌ی generate را نشان می‌دهد. حلقه به چهار نمونه‌ی فیزیکی موازی از tcam_entry تبدیل شده است؛ ساعت، بازنشانی و کلید جست‌وجو میان آن‌ها مشترک است، ولی هر نمونه بخش داده، ماسک، مجوز نوشتن و خط تطبیق مستقل خود را دارد. بنابراین حلقه یک اجرای ترتیبی نرم‌افزاری ایجاد نکرده است.



شکل ۱۴: نمای RTL نسخه‌ی حلقه‌دار با چهار ورودی چهاربیتی موازی

شکل ۱۵ همان ساختار را پس از نگاشت نشان می‌دهد. حضور بافرهای ورودی و خروجی و

تقسیم بردارهای شانزده بیتی داده و ماسک به چهار بخش چهاربیتی تأیید می‌کند که هر ورودی مسیر مستقل دارد و چهار خروجی match_lines حفظ شده‌اند. این تصویر اتصال سخت‌افزار سنتز شده را مستند می‌کند، اما به‌تنهایی مدرک صحت زمانی یا موفقیت مرحله‌ی Fitter نیست.



شکل ۱۵: نمای نگاشت فناوری نسخه‌ی حلقه‌دار و مسیرهای مستقل چهار ورودی

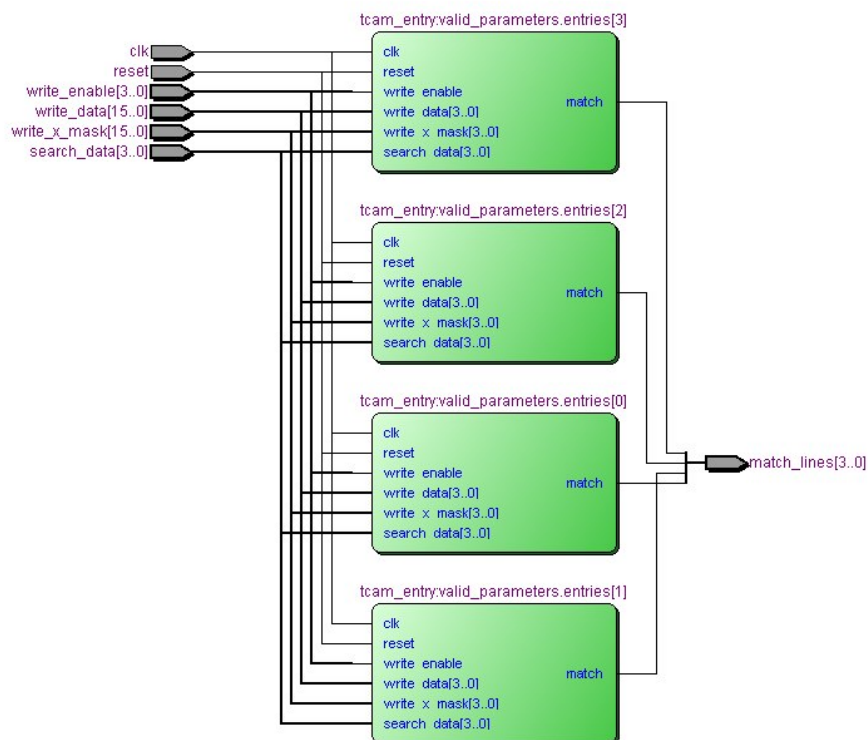
شکل ۱۶ موفقیت مرحله‌ی Analysis & Synthesis برای سطح بالای tcam را ثبت می‌کند. مقدار ۳۶ ثبات دقیقاً از ۱۶ بیت داده، ۱۶ بیت ماسک و چهار بیت اعتبار حاصل می‌شود. استفاده از ۴۶ پایه نیز با رابط موازی چهار در چهار سازگار است.

Analysis & Synthesis Status	Successful - Sat Sep 12 20:39:32 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 5J Web Edition
Revision Name	exp9
Top-level Entity Name	tcam
Family	Cyclone IV GX
Total logic elements	56
Total combinational functions	24
Dedicated logic registers	36
Total registers	36
Total pins	46
Total virtual pins	0
Total memory bits	0
Embedded Multiplier 9-bit elements	0
Total GXB Receiver Channel PCS	0
Total GXB Receiver Channel PMA	0
Total GXB Transmitter Channel PCS	0
Total GXB Transmitter Channel PMA	0
Total PLLs	0

شکل ۱۶: خلاصه‌ی سنتز Quartus برای tcam چهار در چهار

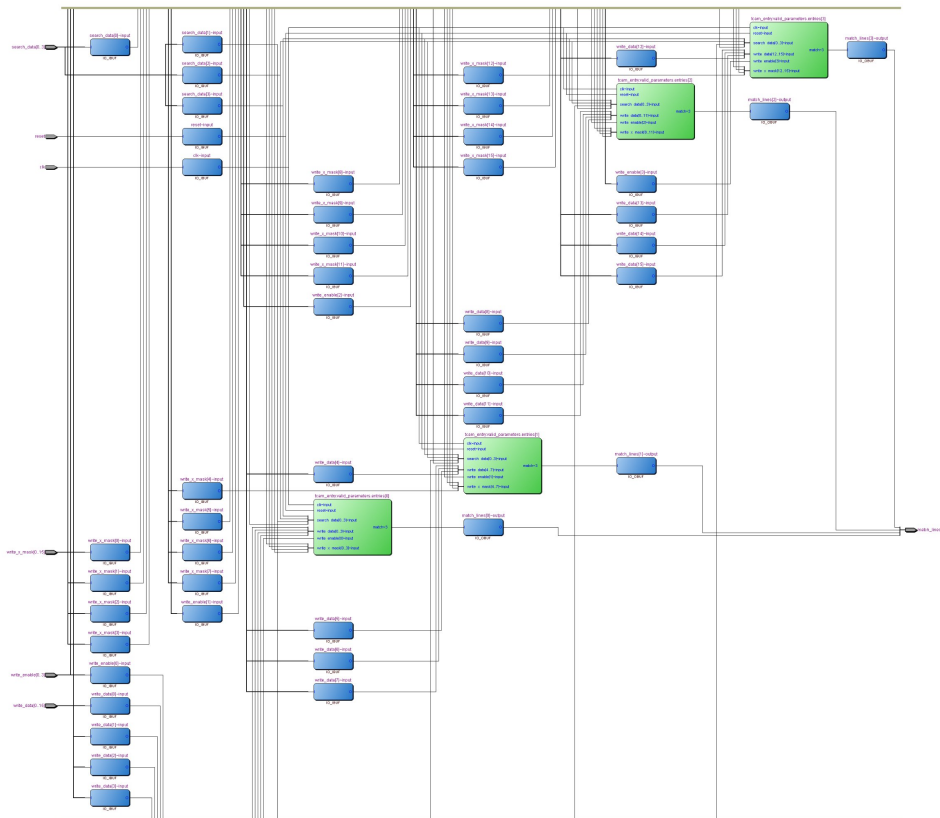
۲-۹ سنتز نسخه‌ی بدون حلقه در پیکربندی چهار در چهار

شکل ۱۷ نشان می‌دهد آرایه‌ی نمونه‌های tcam_no_loop نیز به چهار tcam_entry موازی با همان مرزبندی داده، ماسک و خروجی تبدیل شده است. ترتیب قرارگیری بلوک‌ها در نمودار اهمیتی ندارد؛ اتصال هر نمونه به بیت متناظر مجوز و خروجی، رفتار مستقل ورودی‌ها را حفظ می‌کند.



شکل ۱۷: نمای RTL نسخه‌ی بدون حلقه در پیکربندی چهار در چهار

شکل ۱۸ ثابت می‌کند حذف حلقه از متن کد به کاهش سخت‌افزار منجر نشده است. پس از نگاشت، چهار مسیر ورودی، چهار نمونه‌ی مقایسه‌گر و چهار خط تطبیق همانند نسخه‌ی حلقه‌دار باقی مانده‌اند. این برابری ساختاری انتظار می‌رود، زیرا حلقه‌ی ساخت در زمان تشریح باز می‌شود و منبع سخت‌افزاری زمان اجرا نیست.



شکل ۱۸: نمای نگاهشت فناوری نسخه‌ی بدون حلقه و ساختار موازی معادل

شکل ۱۹ نتیجه‌ی کمی این برابری را ارائه می‌کند. `tcam_no_loop` نیز ۵۶ عنصر منطقی، ۲۴ تابع ترکیبی، ۳۶ ثابت و ۴۶ پایه مصرف کرده است. برابر بودن همه‌ی شمارنده‌های اصلی دو گزارش نشان می‌دهد تفاوت حلقه و آرایه‌ی نمونه‌ها فقط در شیوه‌ی توصیف HDL است.

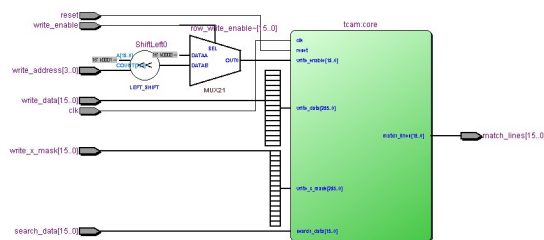
Analysis & Synthesis Status	Successful - Sat Sep 12 20:41:04 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	exp9
Top-level Entity Name	tcam_no_loop
Family	Cyclone IV GX
Total logic elements	56
Total combinational functions	24
Dedicated logic registers	36
Total pins	36
Total pins	46
Total virtual pins	0
Total memory bits	0
Embedded Multiplier 9-bit elements	0
Total GXB Receiver Channel PCS	0
Total GXB Receiver Channel PMA	0
Total GXB Transmitter Channel PCS	0
Total GXB Transmitter Channel PMA	0
Total PLLs	0

شکل ۱۹: خلاصه‌ی سنتز Quartus برای `tcam` بدون حلقه‌ی چهار در چهار

۳-۹ سطح بالای قابل اتصال به بورد برای شانزده در شانزده

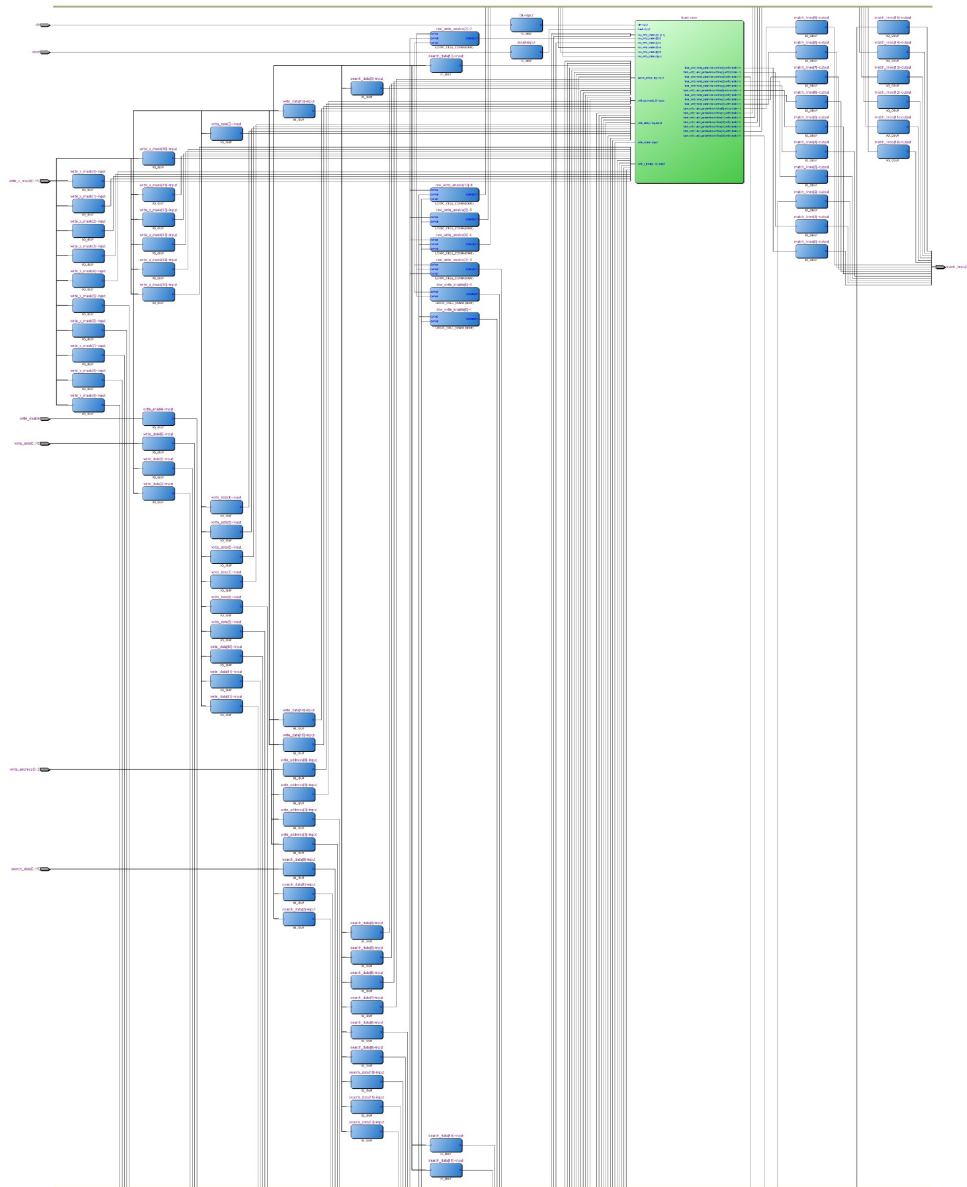
اگر tcam یا tcam_no_loop مستقیماً با پارامترهای شانزده در شانزده به عنوان سطح بالا انتخاب شود، بردار داده‌ی ۲۵۶ بیتی، بردار ماسک ۲۵۶ بیتی، ۱۶ مجوز، کلید ۱۶ بیتی، خروجی ۱۶ بیتی، ساعت و بازنشانی در مجموع ۵۶۲ پایه‌ی سطح بالا می‌خواهند. تغییر حلقه به آرایه این محدودیت را حل نمی‌کند. پیمان‌های tcam_16x16_top ظرفیت داخلی را حفظ می‌کند، اما داده و ماسک مشترک ۱۶ بیتی را همراه یک آدرس چهاربیتی در هر چرخه به یک ورودی می‌نویسد؛ به این ترتیب تعداد پایه‌ها به ۷۱ کاهش می‌یابد.

شکل ۲۰ معماری این تبدیل را نمایش می‌دهد. آدرس نوشتن با یک شیفت به مجوز one-hot شانزده‌بیتی تبدیل می‌شود، داده و ماسک برای اتصال داخلی به هسته تکرار می‌شوند و فقط ورودی انتخاب‌شده آن‌ها را ثبت می‌کند. در نتیجه گذرگاه‌های ۲۵۶ بیتی درون تراشه باقی می‌مانند و دیگر پایه‌ی فیزیکی نیستند، در حالی که جست‌وجوی هر شانزده ورودی همچنان موازی است.



شکل ۲۰: نمای RTL سطح بالای آدرس‌دار و هسته‌ی کامل شانزده در شانزده

شکل ۲۱ نشان می‌دهد ورودی‌های خارجی محدود شده از طریق بافرهای I/O و منطق رمزگشایی به هسته‌ی tcam رسیده‌اند و هر شانزده خط تطبیق نیز تا خروجی حفظ شده است. تراکم اتصال‌ها نتیجه‌ی مقایسه‌ی موازی شانزده کلمه است؛ این نما وجود و اتصال هسته را اثبات می‌کند، ولی برای پیگیری بیت به بیت مناسب نیست.



شکل ۲۱: نمای نگاشت فناوری سطح بالای شانزده در شانزده خط تطبیق

شکل ۲۲ موفقیت تحلیل و سنتز سطح بالای tcam_16x16_top را ثبت می‌کند. ۵۲۸ ثبات دقیقاً با ۲۵۶ بیت داده، ۲۵۶ بیت ماسک و ۱۶ بیت اعتبار منطبق است. در این پیکربندی Quartus مقدار ۸۵۶ عنصر منطقی، ۳۴۴ تابع ترکیبی و ۷۱ پایه گزارش کرده است. کاهش ۴۹۱ پایه نسبت به هسته‌ی موازی، علت اصلی قابل استفاده شدن این سطح بالا برای پکیج محدود برد است.

Analysis & Synthesis Status	Successful - Sat Sep 12 20:59:08 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	exp9
Top-level Entity Name	tcam_16x16_top
Family	Cyclone IV GX
Total logic elements	856
Total combinational functions	344
Dedicated logic registers	528
Total registers	528
Total pins	71
Total virtual pins	0
Total memory bits	0
Embedded Multiplier 9-bit elements	0
Total GXB Receiver Channel PCS	0
Total GXB Receiver Channel PMA	0
Total GXB Transmitter Channel PCS	0
Total GXB Transmitter Channel PMA	0
Total PLLs	0

شکل ۲۲: خلاصه‌ی سنتز Quartus برای سطح بالای شانزده در شانزده

جدول ۲ نتایج واقعی سه اجرای Quartus را مقایسه می‌کند. دو ردیف نخست به پیکربندی چهار در چهار و ردیف سوم به پیکربندی کامل شانزده در شانزده مربوط‌اند.

جدول ۲: مقایسه‌ی نتایج Analysis & Synthesis در Quartus

پایه	ثبات	تابع ترکیبی	عنصر منطقی	سطح بالا
۴۶	۳۶	۲۴	۵۶	4×4 , tcam
۴۶	۳۶	۲۴	۵۶	4×4 , tcam_no_loop
۷۱	۵۲۸	۳۴۴	۸۵۶	16×16 , tcam_16x16_top

این تصاویر موفقیت Analysis & Synthesis را اثبات می‌کنند. چون تصویر مستقلی از Fit-Summary ارائه نشده است، از آن‌ها موفقیت جانمایی نهایی یا بسته‌شدن زمان‌بندی استنتاج نمی‌شود؛ با این حال، شمار ۷۱ پایه در گزارش سنتز نشان می‌دهد علت مستقیم شکست رابط ۵۶۲ پایه‌ای حذف شده است.

۱۰. تصمیم‌های طراحی و ملاحظات پیاده‌سازی روی FPGA

نمایش سه‌حالتی با دو بردار داده و ماسک انتخاب شد، زیرا استفاده از مقدار x زبان Verilog به‌عنوان داده‌ی ذخیره‌شده برای سخت‌افزار واقعی مناسب نیست. در این نمایش، هر بیت TCAM عملاً به یک بیت داده و یک بیت ماسک نیاز دارد. برای ساختار پیش‌فرض، ۲۵۶ بیت داده، ۲۵۶ بیت ماسک

و ۱۶ بیت اعتبار لازم است.

جست‌وجو در همه‌ی ۱۶ ورودی موازی است. این ساختار پاسخ ترکیبی سریع ایجاد می‌کند، اما با افزایش DEPTH و DATA_WIDTH تعداد مقایسه‌کننده‌ها و مسیرهای منطقی افزایش می‌یابد. از آنجا که بسیاری از FPGAها بلوک TCAM اختصاصی ندارند، چنین طرحی معمولاً با ثبات‌ها و LUTها پیاده‌سازی می‌شود. انتخاب بردار match_lines به‌جای priority encoder باعث حفظ همه‌ی تطبیق‌ها و ساده‌ماندن هسته‌ی TCAM شد؛ در صورت نیاز، اولویت‌گذاری می‌تواند به‌عنوان پیمانه‌ای جداگانه پس از match_lines افزوده شود.

reset فقط valid را پاک می‌کند. این تصمیم برای برد مناسب است، زیرا بی‌اعتبارکردن ۱۶ ورودی به پاک‌سازی هم‌زمان تمام ۵۱۲ بیت داده و ماسک نیاز ندارد. همچنین بیت مجوز هر ورودی از تغییر ناخواسته‌ی داده و ماسک ورودی‌های غیرفعال جلوگیری می‌کند.

در دو نسخه‌ی موازی، برای شانزده ورودی شانزده‌بیتی، ۲۵۶ خط داده، ۲۵۶ خط ماسک و ۱۶ مجوز مستقل به سطح بالا منتقل می‌شود. این رابط امکان بارگذاری همه‌ی ورودی‌ها در یک لبه را فراهم می‌کند، اما ۵۶۲ پایه می‌خواهد. سطح بالای tcam_16x16_top این پهنای خارجی را با نوشتن آدرس‌دار به ۷۱ پایه کاهش می‌دهد. هزینه‌ی این تصمیم آن است که بارگذاری همه‌ی شانزده ورودی به شانزده چرخه نیاز دارد، ولی ظرفیت ذخیره‌سازی، منطق تطبیق موازی و اولویت‌بازنشانی بدون تغییر باقی می‌مانند.

۱۱ نتیجه‌گیری

یک TCAM پارامتری با مقدار پیش‌فرض ۱۶ ورودی ۱۶‌بیتی طراحی شد. هر ورودی داده، ماسک و اعتبار مستقل دارد و جست‌وجو به‌صورت ترکیبی در تمام ورودی‌های معتبر انجام می‌شود. رابطه‌ی منطقی استفاده‌شده، بیت‌های ماسک‌شده را حذف و فقط اختلاف بیت‌های اجباری را بررسی می‌کند. هشت آزمون رفتاری، نوشتن هم‌زمان و آدرس‌دار داده‌های متفاوت، ماسک‌های مستقل، حفظ ورودی‌های غیرفعال، بازنشانی و اندازه‌های متفاوت را برای هر سه ساختار تأیید کردند. مثال 01101110 با هر سه الگوی 0110XXXX، X1101XXX و 0X1X11X0 تطبیق یافت و خروجی چندتطبیقی 0111 تولید شد. آزمون یک ثبات صفربیتی و دیگر ابعاد نامعتبر نیز با رد صریح پیکربندی پایان یافت. بررسی Yosys سنتزپذیری را تأیید کرد و نتایج واقعی Quartus نشان داد دو نسخه‌ی چهار در چهار منابع یکسان دارند؛ سطح بالای شانزده در شانزده نیز با ۸۵۶ عنصر منطقی، ۵۲۸ ثبات

و ۷۱ پایه با موفقیت تحلیل و سنتز شد.